

# Live Migration with AMD-V™ Extended Migration Technology

## Table of Contents

|                                                          |   |
|----------------------------------------------------------|---|
| 1. Introduction .....                                    | 1 |
| 2. Virtual Machine Migration .....                       | 2 |
| 3. Determining Processor Features .....                  | 2 |
| 4. VMM Basics .....                                      | 3 |
| 5. Problems with Live Migration .....                    | 3 |
| 5.1 Backward Compatibility Problem .....                 | 3 |
| 5.2 Forward Compatibility Problem .....                  | 3 |
| 6. AMD-V™ Extended Migration Technology .....            | 3 |
| 6.1 Backward Compatibility Assurance .....               | 3 |
| 6.1.1 Controlling Information Returned via CPUID... ..   | 4 |
| 6.1.2 CPUID Override Model Specific Registers .....      | 4 |
| 6.2 Forward Compatibility Assurance .....                | 5 |
| 6.2.1 Feature Disable Bits ..                            | 6 |
| 7. Conclusions .....                                     | 6 |
| 8. References .....                                      | 6 |
| Appendix:                                                |   |
| Using AMD-V Extended Migration on VMware ESX 3.0.1 ..... | 6 |

Virtual Machine migration is a capability being increasingly utilized in today's enterprise environments. With live migration, a Virtual Machine Monitor (VMM) moves a running Virtual Machine (VM) nearly instantaneously from one server to another for a seamless experience to the end user while maintaining guest uptime. However, if the processors running in the computers in the pre and post migration environment are not identical, live migration can result in unexpected behavior of the guest software. Since the introduction of its AMD64 processor technology in 2003, AMD has worked closely with virtualization software developers to define the functionality necessary to ensure that live migration is possible across a broad range of AMD64 processors. This whitepaper discusses the basics of live migration and describes the functionality currently available to enable virtualization software vendors to support live migration across a broad range of AMD64 processors.

## 1. INTRODUCTION

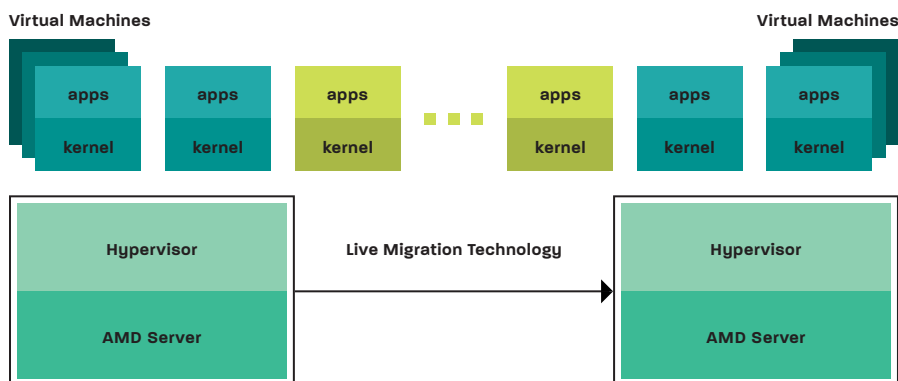
System virtualization is the ability to abstract and pool resources on a physical platform. This abstraction decouples software from hardware and enables multiple operating system images to run concurrently on a single physical platform without interfering with each other.

As a technique, system virtualization has existed for decades on mainframes. In the past, industry standard x86-based machines, with their limited computing resources and challenges in correctly virtualizing the Instruction Set Architecture (ISA)[1,2], did not lend themselves well to the technique. However, with the increase in computing resources of x86-based servers in recent years and with new processor extensions such as those found in AMD-V™[3] technology, x86-based virtualization is seeing widespread adoption and is rapidly becoming a pervasive and integral capability in enterprise environments[4].

Virtualization can increase utilization of computing resources by consolidating the workloads running on many physical systems into virtual machines running on a single physical system. Virtual machines can be provisioned on-demand, replicated and migrated.

Virtual machine (VM) migration, which is the ability to move a VM from one physical server to another under virtual machine monitor (VMM) control, is a capability being increasingly utilized in today's enterprise environments.

In this whitepaper we discuss how AMD-V Extended Migration Technology is designed to enable virtual machine migration between AMD64 processors.

**FIGURE 1 – LIVE VIRTUAL MACHINE MIGRATION ON AMD PROCESSORS**

## 2. VIRTUAL MACHINE MIGRATION

There are many ways to migrate a VM. In *static migration*, the VM is shutdown using OS-supported methods; its static VM image is moved to another VMM and restarted. In *cold migration*, the VM is suspended using OS-supported or VMM-supported methods. The suspended VM image is moved to a VMM on a different machine and resumed. In *live migration*, the VMM moves a running VM instance nearly instantaneously from one server to another. Live Migration allows for dynamic load balancing of virtualized resource pools, hardware maintenance without downtime and dynamic failover support.

For simplicity in the following sections we call software running in the VM “guest software.”

As long as the hardware in the pre and post migration environment is identical, guest software should behave in exactly the same way before and after the migration. It is when guest software runs in a different hardware environment after a migration that certain challenges can arise.

Note that even though a VMM presents a virtual platform to guest software, there could be certain interfaces, depending on VMM design, which guest software can directly use to determine underlying hardware’s capabilities. We will discuss this more in Section 5.

After the reboot/restart following a static migration, guest software should go through its platform discovery phase and be able to adjust to any differences in underlying (virtual) hardware.

Following a cold migration, guest software may continue to maintain an identical view of the pre and post migration hardware. When suspended using OS-supported methods, some operating systems will

re-scan the hardware upon resume. Depending on their policies and the hardware differences between current and previous hardware, the OSes may refuse to resume and require a reboot.

After a live migration, guest software continues to maintain an identical view of the pre and post migration hardware. In this whitepaper we discuss how AMD64 Processors provide support for a VMM to hide differences in software-visible processor features during Live and Cold Migration. For simplicity we will use Live Migration to refer to both migration methods.

## 3. DETERMINING PROCESSOR FEATURES

Software should use CPUID to determine processor features and use this information to select appropriate code paths. CPUID is an x86 instruction that can be executed from kernel and applications. The following example shows CPUID being called from an application.

```
//Example 1: Console application on Windows® XP using CPUID
//instruction to determine
//processor revision and feature information.

#include "stdafx.h"
#include <stdio.h>

int _tmain (int argc, _TCHAR* argv[])

{
    int version;
    int features_1;
    int features_2;

    __asm
    {
        mov eax, 1

        cpuid

        mov version, eax
        mov features_1, ecx
        mov features_2, edx
    }

    printf ("Processor version: 0x%0x features_1: 0x%08x
    features_2: 0x%08x\n", version, features_1, features_2);

    return 0;
}
```

However, a small number of existing commercial software determines the presence of certain processor features using non-standard ways (such as using a try-catch code block to check if certain instructions are supported) or assumes the presence of certain features without testing for them. With such software, it is difficult to provide assurance about their behavior under live migration scenarios. Note that such software is equally likely to fail in non-virtualized environments with different processors.

#### 4. VMM BASICS

To understand the implications of VMM design on VM migration it is useful to briefly review various VMM design alternatives.

It is important to note that for performance reasons, a VMM may only want to interfere in the execution of guest software when the guest code performs privileged operations [1,2].

The first method is called para-virtualization, where the guest's kernel is modified to cooperate with the VMM when performing privileged operations. The second method is called binary translation, where the VMM modifies the guest's binary image at runtime to get processor control when guest software attempts to perform a privileged operation [2]. The VMM can then emulate the privileged operation and return control to guest software. The third method is hardware-assisted virtualization where the VMM uses processor extensions such as AMD-V to intercept and emulate privileged operations in the guest.

#### 5. PROBLEMS WITH LIVE MIGRATION

In the following sections we call the processor in pre-migration environment the "current" processor. The one in post-migration environment is called the "target" processor.

When performing a live migration, a VM is suspended; its current (guest and VMM) state is captured, moved to another VMM, and resumed on it. The VMM preserves the state and identity of virtual devices after the migration. This process happens quickly to present a seamless experience to the end user and to maintain any guest uptime guarantees.

As indicated previously, when hardware in the pre and post migration environment is different, live migration can result in unexpected guest behavior.

After a live migration there is no indication to guest software that it is now running on a different server with potentially different processor features. This is a fundamentally different programming environment which may not interact well with existing software design. For example, a guest application may use CPUID instruction at startup to determine that the processor supports SSE3 instructions, then live migrate to another server and consequently fail when trying to use those instructions if the target processor does not support SSE3.

To help ensure that guest software does not malfunction after a live migration, the virtualization infrastructure should address the following problems.

##### 5.1 BACKWARD COMPATIBILITY PROBLEM

This is the case when the target processor does not implement a feature which is implemented on the current processor.

In such a case, after live migration, use of such a feature may result in unexpected software behavior. For example, use of an instruction not implemented on the target processor will result in a #UD fault.

##### 5.2 FORWARD COMPATIBILITY PROBLEM

This is the case when the target processor implements features not supported on the current processor.

This could be problematic in cases where the target processor uses an opcode (to implement an instruction) which is not defined on the current processor and software on the current processor expects use of this undefined opcode to generate a #UD fault. In such cases, after a live migration, software will observe a different processor behavior, i.e. it will not observe #UD fault upon use of the (previously undefined) opcode.

Fortunately a very small number of existing commercial software depends on such processor behavior. AMD actively works with its software partners to discourage use of such behaviors.

#### 6. AMD-V EXTENDED MIGRATION TECHNOLOGY

Since the introduction of its AMD64 processor technology in 2003, AMD has worked closely with virtualization software developers to define the functionality necessary so that live migration is possible across a broad range of product. AMD-V™ Extended Migration Technology provides various capabilities to address problems associated with Live Migration.

##### 6.1 BACKWARD COMPATIBILITY ASSURANCE

To assure backward compatibility, the VMM can intercept the #UD fault and emulate the faulting instruction. However, this method of emulating the faulting instruction is not generally acceptable as it adds to the complexity of the VMM and may also degrade the performance of the guest.

A simpler and more widely adopted solution by many VMM vendors is to expose to the guest software only those features which are supported on all processors in the resource pool. This essentially means establishing the lowest common denominator of processor features in the resource pool.

### 6.1.1 CONTROLLING INFORMATION

#### RETURNED VIA CPUID

When new features are introduced on AMD processors, their presence is indicated by unique bits in the information returned by the CPUID instruction. Properly written software relies only on this information to determine processor features.

To ensure proper behavior during VM migration, a VMM must control information returned to guest software as a result of CPUID instruction.

A VMM using hardware-assisted virtualization (AMD-V™) (such as Xen HVM or Microsoft® Virtual Server) can use the CPUID intercept to return the appropriate bits whenever guest executes that instruction.

A VMM using binary translation (such as VMware ESX) or para-virtualization (such as Xen) may have to handle this differently depending on whether CPUID was executed by guest's kernel or guest's application. In the case of CPUID instruction executed by the guest's kernel the VMM can binary-translate or para-virtualize that instance of the instruction and return appropriate information to the guest.

For a CPUID instruction executed within the guest application's context (where binary translation or para-virtualization techniques are not feasible), the VMM requires a way to return the appropriate CPUID information to that guest. Note that with binary translation and para-virtualization, it is the user-mode CPUID features that the VMM needs the ability to control. Without this ability all processors in the resource pool must support the same features. This significantly reduces flexibility in utilizing resources in environments supporting Live Migration.

AMD's CPUID Override Model Specific Registers (MSRs), as described in the next section, provide the VMM the ability to control CPUID information returned to guest's applications.

### 6.1.2 CPUID OVERRIDE MODEL

#### SPECIFIC REGISTERS

Starting with Family-0Fh Single-Core AMD Opteron™ Processors, AMD provides CPUID Override MSRs to allow a VMM (or OS) to specify a (subset of) information the CPUID instruction returns. This is accomplished by setting the appropriate MSRs as described below.

**MSR Type:** *CPUID Features Register Override*

**MSR Description:** Provides control over features reported in CPUID function 0000\_0001 in ECX and EDX.

**MSR Index:** C001\_1004h

| Bits  | Description                    |
|-------|--------------------------------|
| 63:32 | Features. R/W, function 1, ECX |
| 31:0  | Features. R/W, function 1, EDX |

**MSR Type:** *Extended CPUID Features Register Override*

**MSR Description:** Provides control over features reported in CPUID function 8000\_0001 in ECX and EDX.

**MSR Index:** C001\_1005h

| Bits  | Description                    |
|-------|--------------------------------|
| 63:32 | Features. R/W, function 1, ECX |
| 31:0  | Features. R/W, function 1, EDX |

**MSR Type:** *Logical Processor count (K8 only)*

**MSR Description:** MSR is provided for backward compatibility with K8 processors.

**MSR Index:** C001\_100Dh

| Bits  | Description                   |
|-------|-------------------------------|
| 63:32 | Reserved                      |
| 31:0  | Logical Processor Count. R/W. |

**MSR Type:** *Processor Name String Registers*

**MSR Description:** These registers hold the CPUID name string in ASCII. The state of these registers is returned by CPUID instruction Functions 8000\_0004, 8000\_0003, 8000\_0002. Each register contains a block of 8 ASCII characters; the least significant byte corresponds to the first ASCII character of the block; the most-significant byte corresponds to the last character of the block. MSR C001\_0030h provides the first block of the name string; MSR C001\_0035h contains the last block of the name string.

**MSR Index:** C001\_0030h – C001\_0035h

| Bits | Description         |
|------|---------------------|
| 63:0 | CpuNameString. R/W. |

When the VMM or OS does not override CPUID return information using this mechanism, the processor returns CPUID information based on the native feature set.

The following code sample shows how a VMM (or OS) can hide reporting of SSE4A instructions on an AMD Opteron™ processor .

```

/* * Example 2: Use MSR C001_1005 to clear bit 6 (SSE4A) reported in
 * ECX after CPUID Function 8000_0001
 */

/*
Read current value of the CPUID Override MSR C001_1005.
After RDMSR completes, EDX:EAX contains the 64bit MSR value.
EDX is loaded with the high 32 bits of the MSR and EAX is loaded
with the low 32 bits. The high 32 bits of this MSR are returned in
ECX after CPUID Function 8000_0001
*/

MOV CX, 0xC0011005h
RDMSR

/*
Clear bit 6 (SSE4A) of EDX register
*/

ANDL EDX, 0xFFFFFBBFh

/*
Write the new EDX:EAX value into CPUID override MSR
*/

WRMSR

```

The following code sample shows how a VMM (or OS) can hide reporting of RDTSCP instruction on Second-Generation AMD Opteron™ processor.

```

/*
 * Example 3: Use MSR C001_1005 to clear bit 27 (RDTSCP) reported in
 * EDX after CPUID Function 8000_0001
 */

/*
Read current value of the CPUID Override MSR C001_1005.
After RDMSR completes, EDX:EAX contains the 64bit MSR value.
EDX is loaded with the high 32 bits of the MSR and EAX is loaded
with the low 32 bits. The low 32 bits of this MSR are returned in
EDX after CPUID Function 8000_0001
*/

/*
Write the new EDX:EAX value into CPUID override MSR.
Second-Generation AMD Opteron™ Processors require a
32 bit password in EDI. Contact AMD to get the password.
*/

MOV EDI, <PASSWORD>

MOV CX, 0xC0011005h
RDMSR

/*
Clear bit 27 (RDTSCP) of EAX register
*/

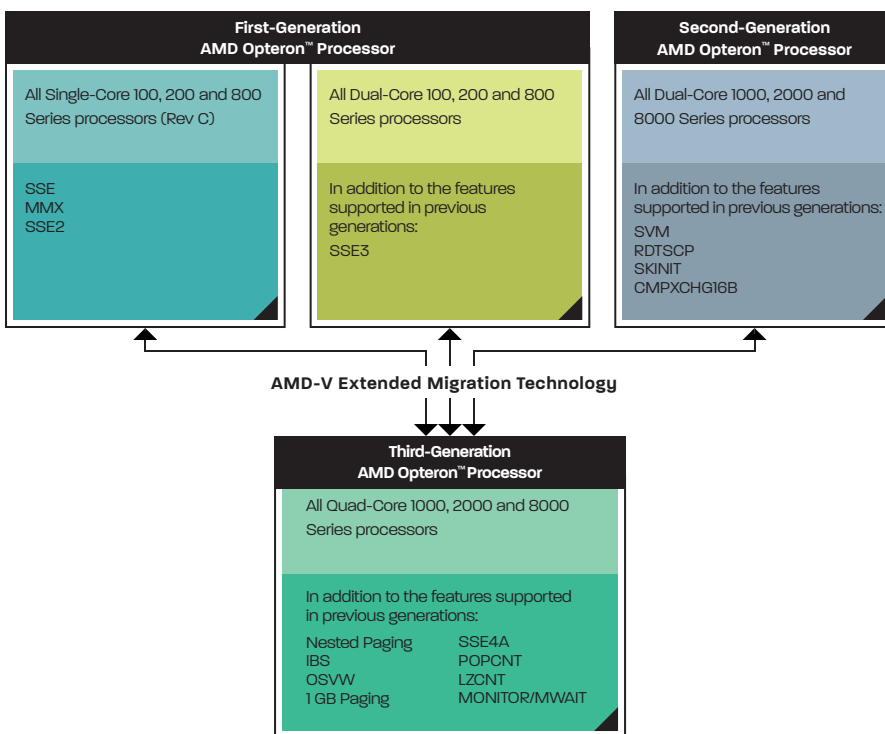
ANDL EAX, 0xF7FFFFFFh

WRMSR

```

It should be noted that overriding CPUID return information using these MSRs does not disable the respective features of the processor. On AMD Opteron™ processors a password must be passed through the EDI register before executing RDMSR and WRMSR to set the CPUID overrides. This password can be obtained from AMD.

## LIVE MIGRATION SUPPORT IN CURRENT AMD PROCESSORS



If guest software does not use CPUID to determine processor features and instead relies on deprecated methods (such as using certain code sequences to detect implementation differences and using the result to identify processor features), these overrides may be of little use. Fortunately, our field surveys indicate that such software is very limited in today's enterprise environments.

## 6.2 FORWARD COMPATIBILITY ASSURANCE

In section 5.2 we discussed forward compatibility problems. To avoid these problems, the VMM needs a method to disable new instructions until all the processors in the resource pool implement them. This is needed to ensure that software will continue to observe the same CPU behavior (such as expected #UD faults) in forward migration scenarios.

### 6.2.1 FEATURE DISABLE BITS

On future processors, AMD may support feature disable bits to disable certain instructions or instruction sets. Such feature disable bits could be exposed through an MSR based interface.

AMD believes that the long term solution for forward compatibility assurance is encouraging developers to write well-behaved, “live-migration friendly” code and adding appropriate support in VMMs to handle forward compatibility problems.

## 7. CONCLUSIONS

In this whitepaper we have described in detail the potential problems that could cause guest software to malfunction after a live migration. We also discussed how AMD64 processors are designed to provide the functionality needed to support the VMM in addressing these problems.

AMD-V Extended Migration Technology allows a VMM to safely migrate a VM between various AMD64 processor revisions. This provides IT administrators an unprecedented flexibility in deploying, maintaining and upgrading servers in live migration environments based on AMD64 processors.

Readers are cautioned about instances of legacy software which rely on non-standard ways to determine processor features. However, such instances are very limited and should diminish over time.

AMD encourages IT administrators to work with their VMM software vendors in determining their VMM's support for AMD-V Extended Migration Technology. The respective vendors can provide guidance when configuring Live Migration resource pools with AMD processors[5].

## 8. REFERENCES

1. POPEK, G. J., GOLDBERG, R. P Formal Requirements for Virtualizable Third Generation Architectures. *Communications of the ACM* 17 (7): 412 –421
2. ADAMS, K., AGESEN, O. Comparison of Software and Hardware Techniques for x86 Virtualization. *ASPLOS*
3. AMD. AMD64 Architecture Programmer's Manual Volume 2
4. IDC. Impact of Virtualization Software on Operating Environments. *IDC Technology Assessment*
5. VMware. VMotion CPU Compatibility Requirements for AMD Processors (<http://kb.vmware.com/kb/1992>)

# Appendix

---

## USING AMD-V EXTENDED MIGRATION ON VMware ESX 3.0.1

AMD expects VMM and Platform vendors will support AMD-V Extended Migration in their future products.

However for demonstration-only purposes and for evaluating the technology before vendors officially support it, AMD has published a binary module at <http://developer.amd.com> which can be loaded and run through GRUB boot-loader before ESX software runs. Please note that AMD has not validated this module extensively and discourages its use beyond an evaluation setup. Please see the associated AMD disclaimer before proceeding to use it.

In the following example, using one Quad-Core AMD Opteron™ processor-based ESX system and one Second-Generation AMD Opteron™ processor-based system running ESX 3.0.1, we show how to live-migrate a VM between them.

We assume that a VMotion environment has been configured according to VMware's guidelines. The following steps are performed on the Quad-Core AMD Opteron™ processor-based system.



**Modify GRUB boot loader**

1. Log into root account on ESX console.
2. move /boot/grub/grub.conf into /boot/grub/grub-2.conf
3. Using a text editor such as 'vi', save the following contents as /boot/grub/grub.conf. Overwrite the value of 'root' parameter with the corresponding value from grub-2.conf. Make sure the value is copied verbatim.

```
# grub.conf for AMD-V™ Extended Migration demo
timeout=10
default=0

title Boot ESX 3.0.1 with Quad-Core AMD Opteron™ CPU feature set
root (hd0,0)
kernel --no-mem-option /vmlinuz-2.4.21-47.0.1.Elvminix
ro root=UUID=e8dcd297-1e2c-4670-84a4-8117d01e160b mem=272M
initrd /initrd-2.4.21-47.0.1.Elvminix.img

title Match CPU feature set to an Second-Generation AMD Opteron™ baseline
root (hd0,0)
kernel /msr_shim root=UUID=e8dcd297-1e2c-4670-84a4-8117d01e160b
msr=0xc0011004,0x2001,0x178bfbff,0x9c5a203a
msr=0xc0011005,0x001f,0xebd3fbff,0x9c5a203a
module /grub/stage2 (hd0,0)/grub/grub-2.conf

title Match CPU feature set to an First-Generation AMD Opteron™ baseline
root (hd0,0)
kernel /msr_shim root=UUID=e8dcd297-1e2c-4670-84a4-8117d01e160b
msr=0xc0011004,0x0001,0x178bfbff,0x9c5a203a
msr=0xc0011005,0x0003,0xe3d3fbff,0x9c5a203a
module /grub/stage2 (hd0,0)/grub/grub-2.conf

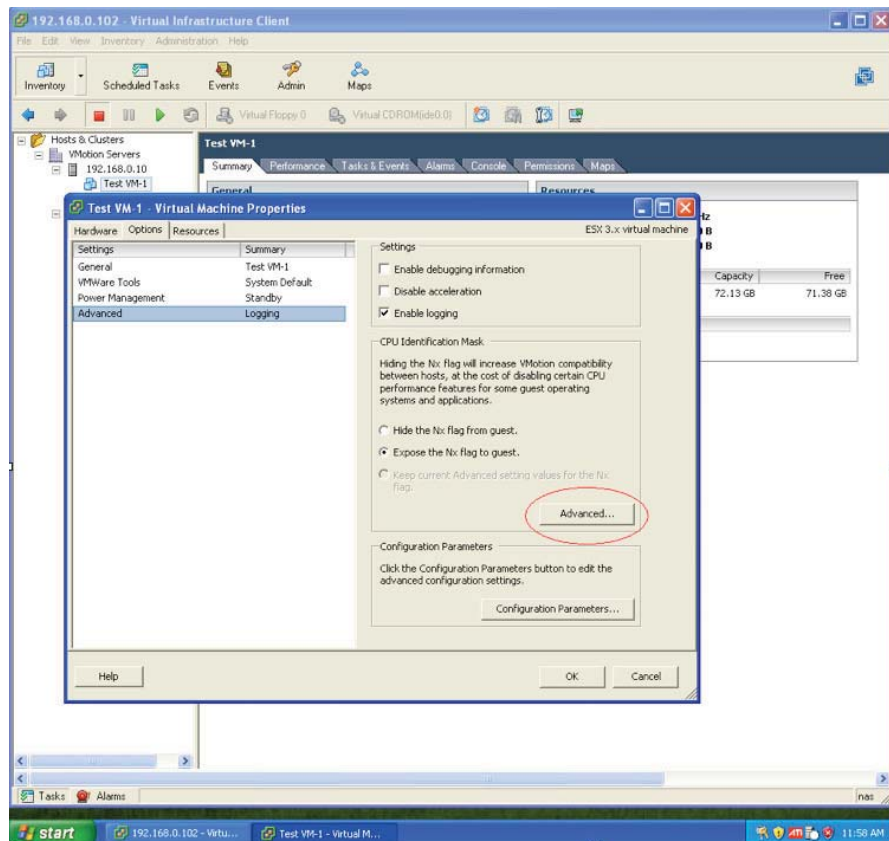
title Match CPU feature set to a Single-Core AMD Opteron™ baseline
root (hd0,0)
kernel /msr_shim root=UUID=e8dcd297-1e2c-4670-84a4-8117d01e160b
msr=0xc0011004,0,0x078bfbff,0x9c5a203a
msr=0xc0011005,0,0xe1d3fbff,0x9c5a203a
module /grub/stage2 (hd0,0)/grub/grub-2.conf
```

**Copy msr\_shim to your ESX installation**

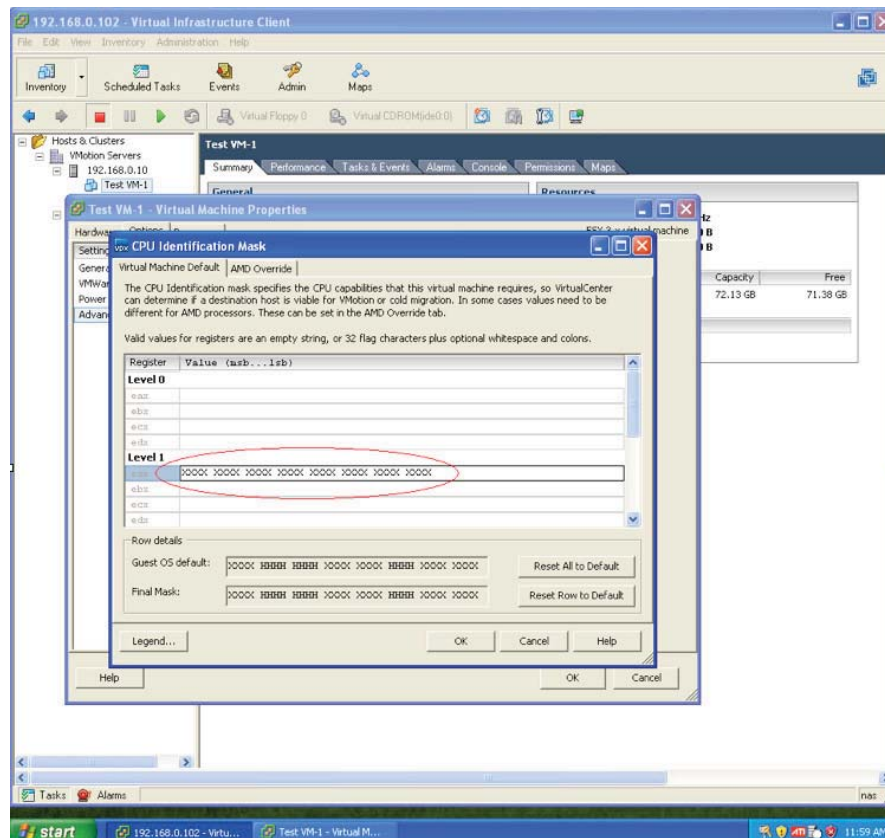
4. Copy msr\_shim (downloadable from developer.amd.com) to the /boot/ directory. This is the same folder where the initrd\* and vmlinuz\* live.

**Configure the VM to ignore CPU Family differences**

5. Using the VI Client, select the VM you want to VMotion and open the Advanced Options dialog box.



6. In the CPU Masks section, Level 1, eax, set the entire mask to XXXX XXXX XXXX XXXX XXXX XXXX



#### Reboot ESX Server

7. When ESX reboots, select the Grub Family/Revision option that matches with the other machine in your VMotion pool, in this example a Second-Generation AMD Opteron™ processor-based system. You will see some messages indicating that MSRs have been programmed and the after a few seconds the Grub options in /boot/grub/grub-2.conf will be displayed.

8. Boot ESX normally.

#### VMotion the VM between Quad-Core AMD Opteron™ processor-based machine to Second-Generation AMD Opteron™ processor-based machine

9. Using the VI Client, start the VM configured in step 5 and drag it to the VMotion target machine.



Advanced Micro Devices  
One AMD Place  
Sunnyvale, CA 94088-3453  
[www.amd.com](http://www.amd.com)